Devups

Tome I — Le socle et le démarrage

Documentation du framework

Deuxième édition — novembre 2026 · Devups 7.1.0

Établie à partir du code du dépôt **spacekola/devups** (github.com/yuricoco/devups-package), version 7.0.0, et de sa suite de tests.

Sommaire

Avant-propos 4
Ce que ce document est 4
Comment il a été produit 4
Comment le lire 4
Ce que ce document ne couvre pas 5

Chapitre 1 — Qu'est-ce que Devups 6
1.1 En une phrase 6
1.2 Les quatre idées qui le caractérisent 6
1.3 Les deux niveaux : composant et module 6
1.4 Prérequis 7

Chapitre 2 — Anatomie d'un projet 8
2.1 L'arborescence racine 8
2.2 La frontière qui compte : `dclass/` contre `src/` 8
2.3 L'intérieur de `dclass/` 9
2.4 L'intérieur d'un module 9
2.5 Les deux fichiers de configuration générés 10

Chapitre 3 — Installation 11
3.1 La séquence officielle 11
3.2 Ce que fait `install` 11
3.3 Les dossiers créés 12
3.4 Le serveur de développement 12
3.5 Vérifier que l'installation tient 12

Chapitre 4 — Configuration 13
4.1 Les fichiers 13
4.2 `dv_env()` 13
4.3 Les variables, par famille 13
4.4 L'ordre de chargement 15
4.5 Le mode production 16

Chapitre 5 — Le cycle de vie d'une requête 17
5.1 Les deux points d'entrée 17
5.2 Les quatre portes 17
5.3 Porte 1 — `App::run()`, les pages publiques 17
5.4 Porte 2 — `WebService::runServe()`, l'API publique 18
5.5 Porte 3 — `Admin::manage()`, les pages du back-office 19
5.6 Porte 4 — `AdminService::manageServe()`, l'API du back-office 19
5.7 `mapFunction()` — le dernier kilomètre 20
5.8 Le cache de routes 20

5.9 Résumé en un tableau 20

Chapitre 6 — La ligne de commande 22

6.1 Le binaire 22
6.2 Cycle de vie 22
6.3 Base de données et migrations 22
6.4 Datacore, droits, langues, documentation 23
6.5 La génération 23
6.6 Le garde de régénération 24
6.7 Ce qu'il faut savoir avant de régénérer 24

Chapitre 7 — Le générateur et le scan du disque 26

7.1 Les trois niveaux de datacore 26
7.2 Le scan 26
7.3 Les trois conséquences à retenir 26
7.4 Créer un module à la main 26
7.5 Ce que `dvups_:update` écrit 27
7.6 Les trois choses que le datacore ne sait pas exprimer 27

Chapitre 8 — Premier module, de bout en bout 28

8.1 Le datacore du composant 28
8.2 Le datacore du module 28
8.3 Le datacore de l'entité 28
8.4 Générer 29
8.5 Les trois gestes manuels 29
8.6 Créer la table 30
8.7 Vérifier 30
8.8 Si quelque chose ne marche pas 30

Chapitre 9 — Les tests 32

9.1 Les trois suites 32
9.2 La base de test 32
9.3 Les classes d'appui 32
9.4 L'écran par écran 32

Avant-propos

Ce que ce document est

Devups est un framework PHP écrit par Aurelien Atemkeng (Spacekola). Ces quatre tomes en sont la documentation de référence pour la **version 7.0.0**, celle qui est publiée sur Packagist sous le nom `spacekola/devups`.

La première édition avait été écrite en lisant le code d'un projet client construit sur Devups 4.3. Elle décrivait fidèlement ce projet-là, avec ses entités métier, ses contournements et ses défauts de l'époque. Cette deuxième édition repart du dépôt du framework lui-même, devenu neutre : plus aucun code applicatif n'y vit, et chaque exemple utilise un projet fictif (`monprojet`, un module `ModuleTask`, des entités `task` et `project`).

Elle est écrite pour trois lecteurs :

- **le développeur qui démarre un projet** avec `composer create-project` et veut aller de l'installation à un premier écran d'administration ;
- **le développeur qui reprend un projet Devups existant** et doit comprendre où les choses se passent avant d'y toucher ;
- **l'auteur et les contributeurs du framework**, pour qui ce document sert de miroir : il dit ce que le code fait réellement, y compris là où ce n'est pas ce qu'il avait l'air de vouloir faire.

Comment il a été produit

Chaque affirmation provient d'une lecture du code de la version 7.0.0, et la plupart sont couvertes par la suite de tests livrée avec le framework (`tests/`, plus de 300 tests unitaires, d'intégration et de bout en bout). Quand un comportement est cité, le fichier qui le porte est nommé — mais plus le numéro de ligne, qui change à chaque version.

Les encadrés suivent une convention constante :

> **Nouveau en 7.0.** Un mécanisme qui n'existait pas, ou qui a changé de forme, depuis la première édition.

> **Piège.** Un comportement réel qui surprend, et qui a déjà coûté du temps sur un projet.

> **À retenir.** Une règle de travail ou une convention.

Comment le lire

Le découpage suit celui de la documentation Laravel : un chapitre explique un concept, montre un exemple complet, puis donne le tableau de référence.

Tome	Contenu
I — Le socle et le démarrage	ce qu'est Devups, l'anatomie d'un projet, l'installation, la configuration, le cycle de vie d'une requête, la ligne de commande, le générateur, un premier module, les tests

Tome	Contenu
II — Les données	le datacore, les entités, le `QueryBuilder`, l'écriture, les transactions, les migrations, la traduction des données
III — L'interface	les routes, les contrôleurs, les styles d'API, les listes, les formulaires, les vues, les langues de l'interface, la sécurité, la documentation d'API
IV — Les modules fournis	les composants du socle, les droits, les journaux, le CMS, les arbres, les statuts, les notifications, les courriels, les dettes connues

Ce que ce document ne couvre pas

- Le générateur Android/Java (`dclass/generator/android/`) : présent, non documenté.
- L'intérieur de `DBAL.php` et de `Dfile.php` : leur contrat public est documenté, pas leur implémentation.
- La couche JavaScript du back-office (`dclass/devupsjs/`) : décrite par son rôle au Tome III, pas en référence d'API.
- Le déploiement en production : les commandes `build` et `deploy` existent, mais aucune procédure de mise en ligne n'est imposée par le framework.

Ce document est destiné à être itéré. Chaque projet devrait y ajouter ce qu'il mesure et corriger ce qu'il trouve faux.

Chapitre 1 — Qu'est-ce que Devups

1.1 En une phrase

Devups est un framework PHP dans lequel l'arborescence des dossiers est le modèle de données, et où un générateur produit, à partir de fichiers JSON de description, l'entité, le formulaire, la liste et les contrôleurs d'un écran d'administration complet — avec un back-office de suivi (rôles et droits, journaux, langues, contenus, notifications) prêt à l'emploi.

1.2 Les quatre idées qui le caractérisent

1. Le datacore. Un dossier n'existe pour le framework que s'il contient un fichier `*Core.json` qui le déclare. Le disque est scanné, et c'est ce scan — pas une table de configuration, pas un fichier de routes — qui établit la liste des composants, des modules et des entités du projet. Un module sans `<module>Core.json` est invisible, silencieusement.

2. Le contrat déclaratif. Une liste de back-office ne s'écrit pas en HTML : elle se déclare en PHP sous forme de tableau (`$datatablemodel`), et un moteur unique (`Datatable.php`) rend toutes les listes. Un formulaire se déclare de la même façon (`$this->fields`), et un moteur unique (`FormFactory.php`) rend tous les formulaires. Changer le moteur change toutes les listes et tous les formulaires d'un coup — c'est la grande force et le grand risque du framework.

3. Trois styles d'API qui cohabitent. Un style générique (`POST /api/create/task`) sert n'importe quelle entité enregistrée sans écrire de contrôleur. Un style REST dédié (`POST /api/task`) passe par un `FrontController` par entité, piloté par annotations. Un style libre (`GET /api/hello` → `helloServe()`) sert les actions qui ne sont pas un CRUD. Les trois vivent sur les mêmes entités et ne se comportent pas pareil : le Tome III les sépare.

4. Doctrine n'est là que pour les métadonnées. `doctrine/orm` lit les annotations `@Column` / `@ManyToOne` et sait créer ou mettre à jour le schéma. Mais aucune requête métier ne passe par lui : toute lecture et toute écriture passent par la couche maison `DBAL` / `QueryBuilder`, dite « Tchutte » (du surnom de l'auteur du socle).

1.3 Les deux niveaux : composant et module

Un projet Devups est découpé en **composants**, eux-mêmes découpés en **modules**.

- Un composant est un domaine. Le framework en livre trois : `devups` (le noyau d'administration), `devupscms` (contenus, arbres, notifications, courriels…) et `devupsfront` (les utilisateurs du site ou de l'application).
- Un module est une fonctionnalité à l'intérieur d'un composant : `ModuleAdmin`, `ModuleTree`, `ModuleUser`…

Un projet ajoute ses propres composants à côté des trois du socle — par exemple `src/monprojet/`. Les composants préfixés `devups*` sont le socle réutilisable : ils sont censés arriver identiques dans tout nouveau projet.

> **Convention, pas contrainte technique.** Rien n'empêche de ranger un module dans un composant auquel il n'appartient pas logiquement. C'est une règle d'hygiène : un module métier va dans le composant du projet, jamais dans un composant `devups*` .

1.4 Prérequis

Élément	Valeur
PHP	**8.1 ou plus** (`composer.json` : `"php": ">=8.1"`), testé en intégration continue sur 8.1, 8.2 et 8.3
Extensions	`pdo_mysql` , `mbstring` , `xml` , `gd` , `zip` , `curl` , `intl`
Base de données	MySQL ou MariaDB
Composer	requis

Dépendances principales (`composer.json`) :

Paquet	Rôle
`doctrine/orm ^2.1.3`	métadonnées d'annotation et création du schéma
`philo/laravel-blade 3.*`	moteur de vues Blade
`firebase/php-jwt ^6.9`	jetons d'authentification de l'API
`phpmailer/phpmailer ^6.6`	envoi de courriels
`mpdf/mpdf ^8.0`	génération de PDF
`guzzlehttp/guzzle ^7.5`	appels HTTP sortants
`maslosoft/addendum`	lecture des annotations de route
`ifsnop/mysqldump-php ^2.9`	sauvegardes de base
`zircote/swagger-php ^4.11`	annotations OpenAPI
`google/apiclient ^2.18`	authentification Google
`minishlink/web-push ^10.0`	notifications Web Push (VAPID)
`barcode-bakery/barcode-1d ^7.0`	codes-barres

En développement s'ajoutent `phpunit/phpunit ^10.5` , `phpstan/phpstan ^1.10` et `symfony/var-dumper` .

L'autochargement est un classmap sur `dclass/` et `src/` . Conséquence pratique : Composer résout une classe par son nom, pas par le composant qui la déclare. Après avoir ajouté, renommé ou déplacé une classe, lancer `composer dump-autoload` .

> **Piège.** Deux projets Devups qui partagent un dossier `vendor/` hérité peuvent voir une entité d'un projet intercepter celle de l'autre. Le symptôme est un `Class 'X' does not exist` qui pointe vers un chemin d'un autre projet. Correctif : `composer dump-autoload` depuis la racine du projet.

Chapitre 2 — Anatomie d'un projet

2.1 L'arborescence racine

```
monprojet/
├── admin/              le back-office (point d'entrée, vues, assets)
├── bin/                binaire Doctrine
├── cache/              vues compilées, droits, langues, arbres, journaux longs
├── config/             constantes, chargement du noyau, configurations générées
├── database/           migrations, seeds, dumps
├── dclass/             LE FRAMEWORK — ne se modifie pas par projet
├── documentation/      documentation d'API et guide développeur servis par l'application
├── lang/               un fichier de traductions par langue (fr.php, en.php…)
├── route/              les quatre portes d'entrée du routage
├── src/                LE PROJET — composants, modules, entités
├── tests/              suites PHPUnit unit / integration / e2e
├── uploads/            fichiers déposés par les utilisateurs
├── vendor/             dépendances Composer
├── web/                le site public (vues, assets, gabarits de courriels)
├── .env.example        modèle de configuration locale
├── composer.json
├── cron.php            tâches planifiées
├── devups              le binaire de ligne de commande
├── header.php          amorçage de l'application publique
├── index.php           point d'entrée public
├── lang.php            les fonctions de traduction t(), tt(), t_lang()
└── phpunit.xml.dist    configuration des tests
```

2.2 La frontière qui compte : `dclass/` contre `src/`

C'est la distinction la plus importante du framework, et la plus facile à enfreindre.

`dclass/` est le framework. Il arrive identique dans tout projet Devups. Y toucher répare un défaut pour un projet et le réintroduit pour tous les autres — ou corrige un comportement sur lequel un autre projet s'appuie. Une correction de `dclass/` se fait dans le dépôt du framework, avec un test, et se diffuse par une nouvelle version.

`src/` est le projet. C'est là que vit tout ce qui est propre à l'application.

> **Règle de travail.** Quand une entité a besoin d'un accesseur que sa classe parente de `dclass/extends/` ne fournit pas, l'accesseur s'ajoute sur l'enfant, dans `src/`, jamais sur le parent.

2.3 L'intérieur de `dclass/`

```
dclass/
├── devups/
│   ├── Core.php                le scan du datacore
│   ├── Genesis.php             le rendu des vues (Blade) et des réponses JSON
│   ├── Log.php                 le journal applicatif (Log::error/warning/info)
│   ├── RateLimiter.php         limitation de fréquence (connexions, codes)
│   ├── Dfile.php               fichiers et images
│   ├── Api.php, ScanDir.php, DvupsTranslation.php
│   ├── Controller/
│   │   ├── Controller.php      contrôleur d'administration (CRUD générique)
│   │   ├── FrontController.php contrôleur d'API (CRUD générique, propriété)
│   │   ├── CrudTrait.php       le CRUD d'administration en six méthodes
│   │   └── ModuleController.php
│   ├── Datatable/              le moteur de listes (Datatable, Lazyloading…)
│   ├── Form/                   le moteur de formulaires (FormManager, FormFactory, Form)
│   ├── Http/                   Router, Request, Route, Response
│   ├── Interfaces/             NotificationSerialize, WorkerCallback
│   ├── Migration/
│   │   └── MigrationRunner.php le moteur de `php devups migrate`
│   ├── model/                  Model, ModelTrait, ModelSerializer, EntityCollection…
│   ├── Resource/views/         vues internes (écran « non monitoré »)
│   └── Tchutte/                QueryBuilder, DBAL, DBALException, DB_dumper, Column
├── devupsjs/                   le JavaScript du back-office
├── extends/                    les classes de base d'entités réutilisables
├── generator/                  le générateur de code, la CLI, OpenAPI, guide
├── lib/
│   ├── annotation/             GET, POST, PUT, DELETE, Api, Auth, MethodView, DDoc
│   ├── addendum/               le lecteur d'annotations
│   └── Util.php, DMail.php, DPdf.php, HZip.php, SimpleXLSX.php
└── dvups_functions.php         fonctions globales (can(), local(), d_url()…)
```

Nouveau en 7.0. `Log.php`, `RateLimiter.php`, `Migration/`, `Tchutte/DBALException.php`, `generator/OpenApiGenerator.php` et `generator/DocGuideGenerator.php`. Ont disparu : `UploadFile.php` (remplacé par `Dfile`), `System_prod.php`, et dans `extends/` les classes propres à un ancien projet (`OrderCore`, `ScanCore`, `CurrencyTrait`, `RoleGuardTrait`, `Item`).

2.4 L'intérieur d'un module

C'est la forme que le générateur produit, et celle qu'un module écrit à la main doit respecter.

```
src/<composant>/<Module>/
├── <module en minuscules>Core.json   LE FICHIER QUI FAIT EXISTER LE MODULE
├── Module<Nom>.php                   la classe du module (écrans de module)
├── <composant>.module<nom>.php       les require des classes du module
├── index.php                         point d'entrée direct du module (généré)
├── services.php                      routes de service historiques du back-office
├── .htaccess                         n'autorise que index.php et services.php
├── Core/<Entité>Core.json            le datacore de chaque entité
├── Entity/<Entité>.php               (+ <Entité>_lang.php si traduite)
├── Controller/<Entité>Controller.php       back-office
├── Controller/<Entité>FrontController.php  API REST dédiée
├── Datatable/<Entité>Table.php
├── Form/<Entité>Form.php
└── Resource/{js,css,views/admin}/
```

Les fichiers de référence sont ceux que produit `php devups core:g:moduleendless <composant>\<Module>` : `index.php`, `services.php`, `.htaccess`, le chargeur `<composant>.module<nom>.php`, les vues `layout` et `overview`. Le générateur fait foi : un module dont ces fichiers ont été modifiés à la main peut être régénéré pour revenir à la forme de référence.

> **Nouveau en 7.0 — le `.htaccess` d'un module.** Il refuse l'exécution directe de tout fichier PHP du module, à l'exception de `index.php` et `services.php`. Et `admin/header.php`, inclus par ces deux points d'entrée, exige une session d'administration pour tout `services.php`, ainsi que pour l'`index.php` des composants `devups` et `devupscms`. L'`index.php` de `devupsfront` reste public : ses routes sont protégées par JWT et `@Auth`.

2.5 Les deux fichiers de configuration générés

`Core::updateDvupsTable()` (dans `Core.php`) écrit deux fichiers PHP à partir du scan du disque :

Fichier	Indexé par	Lu par
`config/dvups_configurations.php`	nom d'entité, et nom pleinement qualifié	le routage d'un écran d'entité, les formulaires, `rights:sync`, la documentation d'API
`config/module_configurations.php`	nom de module	le routage d'un écran de module, la barre latérale du back-office

Chaque entrée d'entité porte `path`, `module`, `name`, `label`, `namespace`, `namespace_name`, `component`. Chaque entrée de module porte `name`, `label`, `icon`, `entities`.

Ces deux fichiers sont générés : la commande `php devups dvups_:update` les réécrit depuis le disque. Ils sont versionnés, parce que l'application les lit à chaque requête. Ils ne se maintiennent pas à la main, sauf pour retirer une entrée orpheline que le scan ne voit plus.

Chapitre 3 — Installation

3.1 La séquence officielle

```
composer create-project spacekola/devups monprojet
cd monprojet

cp config/constante.example.php config/constante.php
cp .env.example .env            # puis renseigner la base, les URLs, les secrets

php devups install              # base, schéma, compte administrateur maître
php devups serve                # http://127.0.0.1:<APP_PORT>/ et /admin/
```

`config/constante.php` est une simple copie du modèle : il lit toutes ses valeurs dans `.env` via `dv_env()` (chapitre 4). Il n'y a donc aucun secret à y écrire.

Pour essayer le framework depuis une copie locale du dépôt, sans passer par Packagist :

```
composer create-project \
  --repository '{"type":"path","url":"/chemin/vers/devups"}' \
  spacekola/devups:dev-master monprojet
```

3.2 Ce que fait `install`

`php devups install` enchaîne, hors production :

1. la création de l'arborescence `cache/` et `database/` si elle manque ;
2. la création de la base si elle n'existe pas ;
3. `doctrine orm:schema:create`, qui crée toutes les tables à partir des entités ;
4. l'exécution de `dclass/generator/config_data.sql`, qui sème les langues, le rôle `admin` et le compte maître, puis la table `configuration` ;
5. la mise en cache des langues ;
6. le **marquage de toutes les migrations existantes comme jouées** : le schéma vient d'être créé à partir des entités actuelles, les rejouer produirait des erreurs (chapitre 6 du Tome II).

Le compte maître est `dv_admin` / `admin`. À la première connexion, le back-office redirige vers `complete-registration` pour le personnaliser.

> **Corrigé en 7.0.** Dans les versions 4.x, `config_data.sql` commençait par ses propres `CREATE TABLE`. Sur une base que Doctrine venait de créer, la première instruction échouait, l'exception était avalée, et aucun administrateur n'était créé — alors que le message de succès s'affichait. Le fichier ne contient plus que des `INSERT`, et la séquence complète est vérifiée de bout en bout (installation dans un dossier vide, puis connexion au back-office).

En production (`APP_PROD=true`), `install` ne crée ni la base ni le schéma : sur un serveur, la base existe déjà et évolue par `php devups migrate`.

3.3 Les dossiers créés

```
cache/
├── views/          vues Blade compilées
└── local/{admin,front}/
database/
├── dump/
├── log/
├── migration/      les migrations (Tome II, chapitre 6)
└── transaction/
```

À l'exécution s'ajoutent sous `cache/` : `rights/` (droits compilés par rôle), `lang/` (langues actives), `trees/` (arbres compilés, Tome IV), `log/` (messages de journal trop longs pour la base) et `ratelimit/`. `php devups init:caches` recrée l'arborescence de base à la demande.

3.4 Le serveur de développement

```
php devups serve
```

Lance le serveur PHP intégré sur le port déclaré par `APP_PORT` (par défaut `:3031` dans `.env.example`), donc `http://127.0.0.1:3031/` pour le site et `http://127.0.0.1:3031/admin/` pour le back-office.

3.5 Vérifier que l'installation tient

Vérification	Attendu
`php devups migrate:status`	aucune migration en attente
`http://<hôte>/admin/`	redirection vers le formulaire de connexion
connexion `dv_admin` / `admin`	redirection vers `complete-registration`
`GET /api/hello`	`{"success":true,"message":"hello devups you made your first apicall"}`
`composer test`	toutes les suites au vert (base de test dédiée, chapitre 9)

Chapitre 4 — Configuration

4.1 Les fichiers

Fichier	Rôle	Suivi par git ?
`.env`	toutes les valeurs propres à une installation : base, URLs, secrets	**non** (ignoré)
`.env.example`	le modèle commenté de `.env`	oui
`config/constante.example.php`	le modèle de `constante.php` : `define()` alimentés par `dv_env()`	oui
`config/constante.php`	la copie locale du modèle	non (ignoré)
`config/env.php`	`dv_env_load()` et `dv_env()`	oui
`config/load_constants.php`	charge `constante.php` (ou le fichier désigné par `DEVUPS_CONSTANTS`)	oui
`config/dependanceInjection.php`	charge les constantes, Composer, tout `dclass/`, puis l'`EntityManager`	oui
`config/Database.php`	ouvre la connexion PDO	oui
`config/dvups_configurations.php`, `config/module_configurations.php`	générés (§2.5)	oui

> **Nouveau en 7.0 — plus aucun secret versionné.** Les versions 4.x versionnaient un `constante.prod.php` qui contenait des valeurs réelles. Désormais, un secret ne vit que dans `.env` ou dans les variables d'environnement du serveur, qui sont prioritaires sur le fichier. Le dépôt ne contient que des valeurs fictives (`change-me`, `example.com`).

4.2 `dv_env()`

```
define('dbname', dv_env('DB_NAME', 'devups'));
define('__prod', dv_env('APP_PROD', false));
```

`dv_env($clé, $défaut)` lit d'abord la variable d'environnement du processus, puis `.env`, puis renvoie le défaut. Les chaînes `true`, `false` et `null` sont converties en vraies valeurs PHP. Pour la suite de tests, la variable `DEVUPS_CONSTANTS` désigne un autre fichier de constantes : c'est ainsi que les tests pointent vers leur base dédiée sans toucher à la configuration locale.

4.3 Les variables, par famille

Les valeurs ci-dessous sont des exemples de forme.

Application

Variable `.env`	Constante	Rôle
`APP_NAME`	`PROJECT_NAME`	nom affiché
`APP_URL`	`__server`	hôte
`APP_PORT`	`__env_port`	`:3031` en développement, vide en production
—	`__env`	`__server . __env_port . '/'`, la racine de toute URL construite
`APP_PROD`	`__prod`	bascule le comportement d'erreur (§4.5)
`FRONT_URL`	`__env_front`	l'application front séparée, si elle existe
`CORS_ORIGINS`	`__csrf_origin`	origines autorisées en CORS, séparées par des virgules
`DEFAULT_LANG`	`__default_lang`, `__lang`	langue par défaut (code ISO à deux lettres)
`PROJECT_ID`	`__project_id`	préfixe des clés de session (concaténé au port)
`ROUTE_CACHE`	`__route_cache`	cache de routes (inactif en pratique, §5.8)

Base de données

Variable	Constante	Rôle
`DB_HOST`, `DB_NAME`, `DB_USER`, `DB_PASSWORD`	`dbhost`, `dbname`, `dbuser`, `dbpassword`	connexion
`DB_DUMPER`	`dbdumper`	si vrai, un dump est pris à chaque requête — à ne jamais activer en production
`DB_TRANSACTION`	`dbtransaction`	journal de transactions

Courriel, jetons, services

Famille	Variables
SMTP	`MAIL_HOST`, `MAIL_PORT`, `MAIL_ENCRYPTION`, `MAIL_USERNAME`, `MAIL_PASSWORD`, `MAIL_FROM`, `MAIL_FROM_NAME`
JWT	`JWT_SECRET`, `JWT_EXPIRE` (`false` = jetons sans expiration, ou une durée comme `+72 hours`)
Tâches planifiées	`CRON_TOKEN`
Notifications FCM	`FCM_PAIR_KEY`, `FCM_USER_ID`, `FCM_PROJECT_ID`, `FCM_AUTH_FILE`
Web Push	`VAPID_PUBLIC_KEY`, `VAPID_PRIVATE_KEY`, `VAPID_SUBJECT`
Google	`GOOGLE_CLIENT_ID`, `GOOGLE_CLIENT_ID_WEB`, `GOOGLE_CLIENT_ID_ANDROID`, `GOOGLE_CLIENT_ID_IOS`, `GOOGLE_CLIENT_SECRET`, `GOOGLE_REDIRECT_URL`
Géolocalisation IP	`IPGEOLOCATION_API_KEY` (facultatif)

Journal, sécurité, langues (nouvelles en 7.0)

Variable	Défaut	Rôle
`LOG_MIN_LEVEL`	`warning`	niveau minimal enregistré par le gestionnaire global d'erreurs
`LOG_RETENTION_DAYS`	`90`	durée de conservation pour `Log::purge()`
`ACTIVATION_CODE_TTL_MINUTES`	`15`	validité d'un code d'activation à 6 chiffres
`RESET_PASSWORD_TOKEN_TTL_MINUTES`	`30`	validité d'un lien de réinitialisation de mot de passe
`RATE_LIMIT_LOGIN_MAX_ATTEMPTS`	`5`	échecs de connexion tolérés par compte…
`RATE_LIMIT_LOGIN_MAX_ATTEMPTS_IP`	`20`	… et par adresse IP
`RATE_LIMIT_LOGIN_WINDOW_SECONDS`	`900`	fenêtre de comptage des échecs
`RATE_LIMIT_CODE_MAX_ATTEMPTS`	`5`	essais d'un code avant invalidation
`RATE_LIMIT_RESEND_MAX_ATTEMPTS`, `RATE_LIMIT_RESEND_WINDOW_SECONDS`	`3`, `3600`	renvois de code
`TRUSTED_PROXIES`	vide	IP de reverse proxy de confiance ; sans elle, `X-Forwarded-For` est ignoré
`LANG_DIR`	`lang`	dossier des fichiers de traduction
`LANG_COLLECT`	vrai hors production	collecte automatique des clés de traduction manquantes

Constantes dérivées (non configurables) : `ROOT`, `UPLOAD_DIR`, `admin_dir`, `web_dir`, `CLASSJS`, `__admin`, `__front`, et les clés de session `ADMIN` (`__project_id . '_devups'`), `CSRFTOKEN`, `dv_role_navigation`, `dv_role_permission`. Le préfixe `__project_id` permet à deux projets Devups servis depuis le même hôte de ne pas partager leurs sessions.

4.4 L'ordre de chargement

`config/dependanceInjection.php` est le seul point commun à tous les points d'entrée (site, back-office, CLI, tests). Il fait, dans cet ordre :

1. `require config/load_constants.php` — donc `constante.php`, donc `.env` ;
2. `require vendor/autoload.php` ;
3. `require config/Database.php` ;
4. `require`, un par un, les fichiers de `dclass/` — l'ordre est significatif (`ModelSerializer` avant `ModelTrait` avant `Model`, `QueryBuilder` avant `Controller`) ;
5. `$em = DBAL::getEntityManager()` — l'`EntityManager` Doctrine, global.

Puis `header.php` (application publique) ajoute :

```
session_start();
require __DIR__ . '/config/dependanceInjection.php';
require __DIR__ . '/lang.php';           // traductions de la langue courante
require 'src/requires.php';              // tous les modules du projet
Log::installGlobalHandler();             // gestionnaire global d'erreurs
require 'route/WebService.php';
require 'route/App.php';
$dlangs = Dvups_lang::cachedActiveDlangs();   // langues actives, depuis le cache
$global_config = require ROOT . 'config/dvups_configurations.php';
```

`admin/header.php` fait l'équivalent pour le back-office, puis restaure la session d'administration et applique la garde de session des points d'entrée de module (§2.4).

> **`src/requires.php` est la liste des modules du projet.** Un nouveau module doit y être ajouté à la main, sinon : `Fatal error: Class XController not found`.

> **Le gestionnaire global d'erreurs n'est installé que par les deux `header.php`**, jamais par `dependanceInjection.php`. La CLI et PHPUnit gardent donc leur propre gestion d'erreurs.

4.5 Le mode production

`__prod` change le comportement sur quatre points :

Situation	`APP_PROD=false`	`APP_PROD=true`
erreur non rattrapée	détail affiché (page, JSON ou texte selon le contexte)	réponse générique, détail seulement dans le journal
route introuvable (page)	message de développement	`Genesis::render('404')`
route introuvable (API)	message de développement en JSON	`{"success":false,"message":"404 : ... service note found"}`
collecte des clés de traduction	active	inactive

> **`dv_dump()` n'est pas un journal.** C'est `dump($args); die(1);`, en développement comme en production : la requête s'arrête. Pour signaler un problème sans tuer la requête, utiliser `Log::warning()` ou `Log::error()` (Tome IV, chapitre 3).

Chapitre 5 — Le cycle de vie d'une requête

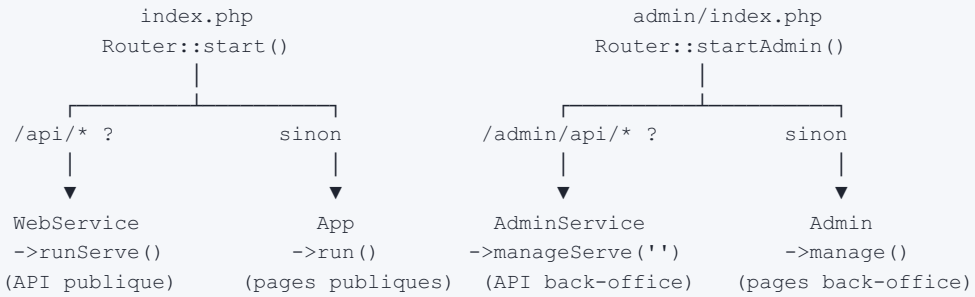
C'est le chapitre à lire avant tous les autres : la plupart des surprises de ce framework viennent de ce que quatre dispatchers cohabitent, et qu'ils ne suivent pas exactement les mêmes règles.

5.1 Les deux points d'entrée

Fichier	Appelle
`index.php`	`Router::start()` — l'application publique et son API
`admin/index.php`	`Router::startAdmin()` — le back-office et son API

`Router::getPathInfo()` établit le chemin en essayant, dans l'ordre : `$_SERVER['PATH_INFO']`, puis `REQUEST_URI` moins le répertoire de base, puis `$_GET['url']`, puis la chaîne de requête. La partie après `?` est retirée.

5.2 Les quatre portes



Les quatre classes vivent dans `route/` et héritent de `Router`.

5.3 Porte 1 — `App::run()`, les pages publiques

1. **Langue dans l'URL.** Si le premier segment est le code ISO d'une langue active (`$dlangs`), il est retiré du chemin et la langue est positionnée. `/fr/contact` et `/contact` mènent au même endroit, dans deux langues.
2. **Nom de méthode.** Le segment suivant est découpé sur `-` et `_`, chaque morceau prend une capitale initiale, et `View` est ajouté : `/mon-compte` → `MonCompteView()`, `/` → `HelloView()`.
3. Si la méthode n'existe pas sur `App`, le tableau `$append_routes` est consulté — le seul mécanisme de route nommée :

```
// route/App.php
public $append_routes = [
    '/imprimer-facture' => 'Invoice@printInvoice',
];
```

1. Sinon : 404 en production, message de développement sinon.
2. `mapFunction('App', $function)` (§5.7).

5.4 Porte 2 — `WebService::runServe()`, l'API publique

a. CORS. L'origine de la requête est comparée à `__csrf_origin`. Si elle y figure (ou vaut `null`), les en-têtes `Access-Control-Allow-Origin` et `-Credentials` sont émis. Les en-têtes autorisés sont codés en dur dans `Router.php` : `Authorization, Auth, X-Customer-Auth, Columns, Dlang, common, Content-Type, X-Requested-With`. Une requête `OPTIONS` répond 200 et s'arrête.

> **Piège de développement.** Si le serveur de développement du front choisit un port absent de `CORS_ORIGINS`, le navigateur n'affiche qu'un « Failed to fetch ». Ajouter l'origine à `CORS_ORIGINS`.

b. Deux en-têtes lus comme des paramètres. `Columns` (un JSON `{"relation":["col1","col2"]}` qui déclenche un `with()`, Tome II) et `Dlang` (force la langue de la réponse).

c. L'authentification, inconditionnellement. `(new Auth())->execute()` est appelé avant tout dispatch. S'il réussit, `Auth::$user_id` est peuplé. S'il échoue, rien ne se passe : l'absence de jeton n'est pas une erreur à ce stade ; elle le devient plus loin si la méthode visée l'exige.

d. Le dispatch. Le chemin, privé de `/api/`, est découpé sur `/` :

Condition	Dispatch	Style
≥ 2 segments et segment 0 ∈ `create`, `upload`, `detail`, `read`, `update`, `delete`, `lazyloading`	`systemServe()`	générique
segment 0 est une clé de `$public_access`	`FrontController::frontServe()`	REST dédié
sinon	`<Segment>Serve()` sur `WebService`	libre

```
POST /api/create/task          → systemServe()     générique
GET  /api/task/42              → TaskFrontController::detail(42)
GET  /api/hello                → helloServe()      libre
POST /api/accept-invitation    → acceptInvitationServe()
```

e. Si la méthode n'existe pas : `$append_routes`, puis 404.

5.5 Porte 3 — `Admin::manage()`, les pages du back-office

`Router::manage()` positionne `Router::$admin_route = true` — un drapeau qui a des conséquences de sécurité réelles (Tome III) — puis :

1. **Session obligatoire.** Si `$_SESSION[ADMIN]` est absent et que le chemin n'est pas `connexion-action`, redirection vers la page de connexion.
2. Le chemin est découpé ; `admin` est ajouté en tête s'il manque.
3. Le nombre de segments décide :

Segments	Interprétation	Conséquence
≥ 5	`/admin/<composant>/<module>/<entité>/<action>`	`Controller::views()` — le CRUD générique, après vérification des droits
≥ 3	`/admin/<composant>/<module>`	`Controller::viewsModule()` — l'écran du module
sinon	`/admin/<page>`	méthode `<Page>View()` sur `Admin`

```
/admin/                                  → DashboardView()
/admin/monprojet/ModuleTask              → écran du module
/admin/monprojet/ModuleTask/task/list    → TaskController::listView()
```

C'est ce mécanisme qui donne à une entité générée un écran de liste, de détail et de formulaire sans qu'une ligne de contrôleur d'écran soit écrite — à condition que l'entité figure dans `dvups_configurations.php`.

5.6 Porte 4 — `AdminService::manageServe()`, l'API du back-office

Forme	Dispatch
2 segments, segment 0 ∈ les sept verbes génériques	`systemServe()`
2 segments, `<entité>/<action>` (`/admin/api/task/create`)	`Controller::serve()` → `Request::Controller()`
sinon	`<Path>Service()` sur `AdminService`

Noter la différence de suffixe : `View` pour une page, `Serve` pour l'API publique, `Service` pour l'API du back-office.

> **Nouveau en 7.0 — la garde des droits.** `Request::Controller()` est le point de passage unique de toute action CRUD d'entité, qu'elle arrive par une page (§5.5) ou par l'API du back-office. Avant d'appeler la méthode, il traduit son nom en action (`createAction` → `create`, `listView` et `formView` → `read` …) et appelle `can($action, $entité)` . Refus : 403. Le Tome IV, chapitre 2, décrit le système de droits.

> **Corrigé en 7.1 — le garde de session de `manageServe()`.** Le contrôle de session en tête de `manageServe()` était jusqu'ici un bloc entièrement commenté (aucun refus réel) : `Admin`, `ModuleConfig`, `ModuleLog`, `ModuleUser` (et tout module généré sur le même patron) n'avaient donc aucun garde qui leur soit propre à ce point d'entrée partagé — protégés en pratique seulement par `admin/header.php` (par nom de script/chemin) pour les points d'entrée historiques, ou par le constructeur d'`AdminService`. `manageServe()` refuse désormais lui-même (401 JSON) toute requête sans session d'administration, avec les mêmes exceptions que `manage()` (`connexion-action`, `public_access`) — une méthode `xxxService()` écrite à la main sur n'importe quelle classe héritant de ce point d'entrée est donc protégée par construction, plus seulement par le script qui l'inclut.

5.7 `mapFunction()` — le dernier kilomètre

`Router::mapFunction($classname, $function)` est appelée par trois des quatre portes. Elle :

1. réfléchit la méthode cible (`ReflectionAnnotatedClass`) ;
2. lie les paramètres par nom : une méthode `detailView($id)` reçoit automatiquement `?id=...` ;
3. contrôle `@Auth` : si la méthode porte l'annotation, `Auth::$user_id` doit être peuplé, sinon exception ;
4. cherche une annotation du verbe HTTP courant (`@GET`, `@POST`…), sinon `@Api`, pour extraire les paramètres de chemin ;
5. appelle la méthode.

> **Divergence entre dispatchers.** Si aucune annotation de verbe ni `@Api` n'est trouvée, `mapFunction()` continue quand même : une méthode `xxxServe()` ou `xxxView()` sans annotation est appelable. Ce n'est pas vrai du style REST dédié : `Request::FrontController()` répond **405** à une méthode de `FrontController` sans annotation de verbe. Une méthode publique de `FrontController` sans annotation n'est donc atteignable par aucune route.

5.8 Le cache de routes

Si `__route_cache` est vrai, `runCache()` consulte `cache/routes.php` avant le dispatch normal. Mais `Router::cacheRoute()`, qui devrait écrire ce fichier, commence par `return null;`. Le mécanisme est complet côté lecture et mort côté écriture : laisser `ROUTE_CACHE=false`.

> **Confirmé en 7.1.** Audit explicite (ÉTAPE E.4) : `cacheRoute()` a son propre `return null;` précoce et **inconditionnel** (atteint même si `__route_cache` est vrai), et `runCache()` ne dépasse jamais son `if (!__route_cache) return null;` puisque cette constante vaut toujours `false` par défaut. Les deux méthodes sont désormais commentées en ce sens dans le code plutôt que supprimées (plusieurs sites d'appel dans `Request.php`, jamais exercés par aucun test) — rien n'a changé fonctionnellement, le constat ci-dessus reste exact.

5.9 Résumé en un tableau

Porte	Classe	Convention de méthode	405 sur annotation absente	Contrôle d'accès
Pages publiques	`App`	`<Path>View()`	non	`@Auth` si posé

Porte	Classe	Convention de méthode	405 sur annotation absente	Contrôle d'accès
API publique	`WebService`	`<Path>Serve()`, générique, ou `FrontController`	oui, pour `FrontController` seulement	JWT, `Auth::$restrictions`, `@Auth`
Pages back-office	`Admin`	`<Path>View()`, ou CRUD générique par nombre de segments	non	session + `can()`
API back-office	`AdminService`	`<Path>Service()`, ou générique	non	`can()` pour les entités ; à la main pour un `xxxService()`

Chapitre 6 — La ligne de commande

6.1 Le binaire

```
<?php
// monprojet/devups
include('dclass/generator/builder.php');
```

Tout se passe dans `dclass/generator/builder.php`, qui analyse `$argv`.

```
php devups <commande> [cible] [options]
```

Sans argument reconnu, l'aide est affichée.

6.2 Cycle de vie

Commande	Effet
`install`	crée la base, le schéma, le compte maître, marque les migrations comme jouées (§3.2)
`database:create`	crée la base et le schéma seuls
`init:caches`	crée `cache/` et `database/` avec leurs sous-dossiers
`serve`	lance le serveur PHP de développement
`build`	copie `src/` et `dclass/` dans `build/` et produit une archive zip
`deploy`	décompresse cette archive à la racine

Pour les tests, utiliser `composer test` (chapitre 9) plutôt que l'ancienne commande `php devups tests`.

6.3 Base de données et migrations

Commande	Effet
`migrate`	joue les migrations en attente de `database/migration/`
`migrate --dry-run`	affiche le SQL sans l'exécuter
`migrate --step`	un lot distinct par migration
`migrate --force`	saute la confirmation d'une migration destructive
`migrate:status`	liste les migrations jouées et en attente
`schema:update --dump-sql`	affiche le SQL que Doctrine appliquerait pour aligner le schéma sur les entités
`schema:update --force`	l'applique
`transaction:rollback <id>`	rejoue un dump de `database/dump/` et annule une transaction journalisée
`location:seed`	charge la liste des pays (idempotent)

> **`migrate` ou `schema:update` ?** `migrate` est la voie normale pour faire évoluer une base existante, sur tous les environnements : les migrations sont versionnées, jouées dans l'ordre et tracées. `schema:update --dump-sql` reste l'outil de diagnostic qui dit si le schéma réel correspond aux entités. Le Tome II, chapitre 6, détaille la différence.

> **Piège Doctrine.** `schema:update --force` peut répondre « updated successfully » sans rien appliquer sur un renommage d'index provoqué par l'ajout d'une relation à une table déjà en base. `--dump-sql` propose alors la même séquence en boucle. Contournement : écrire ce SQL dans une migration et la jouer avec `migrate`.

6.4 Datacore, droits, langues, documentation

Commande	Effet
`dvups_:update`	rescanne `src/`, réécrit les deux fichiers de configuration (§2.5) et met à jour les composants, modules et entités du rôle maître
`dvups_:admin`	rejoue `config_data.sql`
`rights:sync`	complète le catalogue des actions (`dvups_action`) depuis les datacores et rafraîchit le menu de chaque rôle (Tome IV)
`lang:scan`	parcourt le code, repère les appels `t('...')` à clé littérale et complète les fichiers `lang/*.php`
`lang:missing`	liste, par langue, les clés encore non traduites
`api:doc`	écrit un instantané `openapi.json` à la racine (routes front)
`doc:guide`	écrit une page HTML par module et entité sous `documentation/guide/`
`entity:g:core <cible> [--sync]`	(re)génère le `<Entité>Core.json` d'une entité existante
`entity:g:postmandoc <cible>`	génère une collection Postman

`dvups_:update` est à lancer après toute création ou suppression de module ou d'entité, puis `rights:sync` pour que les nouvelles actions soient attribuables aux rôles.

6.5 La génération

La cible s'écrit `<composant>\<Module>\<entité>`, avec des barres obliques inversées :

```
php devups core:g:crud monprojet\ModuleTask\task
```

Commande	Génère
`core:g:component <composant>`	tout un composant, depuis son `<composant>Core.json`
`core:g:module <composant>\<Module>`	la coquille d'un module
`core:g:moduleendless <composant>\<Module>`	toutes les pièces d'un module : CRUD de chaque entité, `index.php`, `services.php`, chargeur, ressources
`core:g:crud <cible>`	entité + formulaire + liste + contrôleur d'administration

Commande	Génère
`core:g:entity <cible> [--lang]`	`Entity/<Entité>.php` (ou la classe `_lang`)
`core:g:form <cible>`	`Form/<Entité>Form.php`
`core:g:table <cible>`	`Datatable/<Entité>Table.php`
`core:g:controller <cible>`	`Controller/<Entité>Controller.php`
`core:g:frontcontroller <cible>`	`Controller/<Entité>FrontController.php`
`core:g:views <cible>`	les vues Blade
`core:g:formwidget`, `core:g:viewswidget`, `core:g:genesis`, `core:g:dependencies`	pièces spécialisées
`core:g:moduledependencies`, `core:g:moduleindex`, `core:g:moduleservices`, `core:g:moduleressources`	les pièces d'un module, séparément

`core:g:crud` ne génère pas de `FrontController`. Il faut `core:g:frontcontroller`, puis décommenter sa ligne `require` dans `<composant>.module<nom>.php` : elle est générée commentée.

6.6 Le garde de régénération

`GeneratorFileGuard` (`dclass/generator/GeneratorFileGuard.php`) protège le travail manuel : avant d'écrire un fichier qui existe déjà, il le renomme en une version horodatée, et ne génère que si le renommage a réussi. À la fin, un récapitulatif liste ce qui a été renommé ou n'a pas pu l'être. L'option `--force`, cherchée n'importe où dans la ligne de commande, désactive le renommage et écrase.

6.7 Ce qu'il faut savoir avant de régénérer

1. Régénérer une entité existante pouvait dupliquer ses `require`. `core:g:crud` sur une entité déjà générée pouvait ajouter un second bloc de `require` à la fin de `<composant>.module<nom>.php`. Ce sont des `require` et non des `require_once` : le résultat était un `Cannot redeclare class X` qui dépend de l'ordre de chargement.

> **Corrigé en 7.1 (D7).** `__Generator::dependencies()` vérifie désormais si le `require` de l'entité est déjà présent avant d'ajouter un bloc en mode "append" — régénérer la même entité deux fois de suite n'écrit plus rien de nouveau si son bloc existe déjà. Relire quand même le fichier de module après une régénération manuelle reste une bonne pratique pour toute autre modification faite à la main.

2. Une entité éditée à la main pour une contrainte sort de la régénération. Une `@UniqueConstraint`, une identité composite : tout cela est écrasé par `core:g:crud`. L'entité passe alors en édition manuelle permanente.

3. Le `nullable` d'une relation était ignoré par le générateur. `BackendGenerator.php` interpolait `nullable` pour une colonne simple, mais pour une relation ne produisait que `@JoinColumn(onDelete="...")` — la clé étrangère restait toujours NULLable, quoi que dise le datacore.

> **Corrigé en 7.1 (D6).** Une relation déclarée `"not"` nullable dans le datacore produit désormais `@JoinColumn(onDelete="...", nullable=false)`. Une relation `"default"` (le cas courant) reste inchangée.

Corrigé en 7.0 — le `update()` généré écrit vraiment. Dans les versions 4.x, `Model::hydrate()` posait `$this->dvold = $this`, une référence et non une copie. La mise à jour ne détectait donc jamais de changement et répondait `Empty parameters` avec un 200. `hydrate()` pose désormais un vrai clone : le gabarit `hydrateWithJson(new X($id), ...)` et l'`updateAction()` du back-office fonctionnent sans retouche. Des tests de régression le couvrent.

Chapitre 7 — Le générateur et le scan du disque

7.1 Les trois niveaux de datacore

Niveau	Fichier	Emplacement
Composant	`<composant>Core.json`	racine du composant
Module	`<module en minuscules>Core.json`	racine du module
Entité	`<Entité>Core.json`	dans `Core/` du module

Le Tome II décrit le contenu de ces fichiers. Ce chapitre décrit comment ils sont lus.

7.2 Le scan

`Core::buildOriginCore()` parcourt `src/` et, pour chaque dossier, appelle `Core::findprojectcore()` :

- `findprojectcore($dir, $file)` : si `$dir/<file en minuscules>Core.json` n'existe pas, retourne `[]` — le composant n'existe pas pour le framework. Sinon, lit ce fichier, scanne les sous-dossiers par `findmodulecore()` et **écrase** `listmodule` par le résultat.
- `findmodulecore($dir, $file)` : même principe pour un module ; appelle `findentitycore($dir . "/Core")` et pose le résultat dans `listentity`.
- `findentitycore($dir)` : décode en JSON chaque fichier de `Core/`.

7.3 Les trois conséquences à retenir

1. **`listmodule` dans un `<composant>Core.json` est ignoré.** Il est écrasé par le scan des sous-dossiers.
2. **Un module sans son `*Core.json` est invisible, sans erreur.** Ni `dvups_:update`, ni `schema:update`, ni la documentation ne le voient.
3. **Les `<Entité>Core.json` ne pilotent pas Doctrine.** Une fois le module trouvé, tout fichier annoté `@Entity` dans son dossier `Entity/` est pris en compte par Doctrine, qu'il ait un datacore ou non. Les datacores servent à la génération, à `rights:sync` et à la documentation.

7.4 Créer un module à la main

1. `src/<composant>/<Module>/<module>Core.json` → le module existe.
2. Les entités dans `Entity/`, annotées `@Entity`.
3. Une migration qui crée les tables (Tome II, chapitre 6), puis `php devups migrate`.
4. `require '<composant>/<Module>/<composant>.module<nom>.php';` dans `src/requires.php` → les classes sont chargées.
5. `php devups dvups_:update` → l'entité apparaît dans le back-office.

6. `php devups rights:sync` → ses actions sont attribuables aux rôles.
7. Pour l'API REST dédiée : enregistrer l'entité dans `$public_access` de `route/WebService.php` (Tome III).

7.5 Ce que `dvups_:update` écrit

`Core::updateDvupsTable()` construit les deux tableaux de configuration (§2.5), les écrit par `Util::arrayToPhpFile()`, puis met à jour le rôle maître (composants, modules, entités, menu).

`Util::arrayToPhpFile()` écrit de façon atomique et assainie : le nom de fichier doit correspondre à `^[A-Za-z0-9_\-]+(\.[A-Za-z0-9_\-]+)*$`, ce qui interdit `../`, et l'écriture passe par un fichier temporaire suivi d'un `rename()`. Aucun lecteur ne voit jamais un fichier à moitié écrit. C'est la même fonction qui écrit les caches d'arbres et de statuts (Tome IV).

7.6 Les trois choses que le datacore ne sait pas exprimer

Besoin	État	Solution
contrainte d'unicité	aucune notion dans le format	`@UniqueConstraint(name=..., columns={...})` à la main
relation obligatoire	`"nullable":"not"` ignoré sur une relation (§6.7)	`nullable=false` à la main dans `@JoinColumn`
identité composite	aucune notion	`@Id @ManyToOne` à la main, pas de colonne `id`

Dans les trois cas, l'entité passe en édition manuelle permanente.

> **Sur la contrainte d'unicité.** Nommer la contrainte à l'identique de l'index physique déjà en base. Doctrine le reconnaît alors comme sien et ne propose plus de `DROP INDEX` dans `--dump-sql`. À l'inverse, `@Column(..., unique=true)` sans nom fait générer par Doctrine son propre nom (`UNIQ_...`), différent de l'existant.

> **Piège de docblock.** Le lecteur d'annotations de Doctrine ne lit que le docblock immédiatement adjacent à `class Xxx`. Intercaler un docblock de prose (`/** ... */`) entre l'annotation et la classe fait échouer toute lecture de méta-don-
> nées (« is not a valid entity or mapped super class »). Une explication va dans un commentaire ordinaire `/* ... */`, placé avant.

Chapitre 8 — Premier module, de bout en bout

Ce chapitre assemble les précédents sur un exemple complet : un module `ModuleTask` dans un composant `monpro-jet`, avec une entité `task` liée à une entité `project` déjà existante.

8.1 Le datacore du composant

`src/monprojet/monprojetCore.json` :

```
{
  "name": "monprojet",
  "template": "adminv2",
  "listmodule": []
}
```

8.2 Le datacore du module

`src/monprojet/ModuleTask/moduletaskCore.json` :

```
{
  "name": "ModuleTask",
  "label": "Tâches",
  "icon": "fas fa-fw fa-check-square"
}
```

`label` et `icon` alimentent la barre latérale du back-office. Sans eux, le nom du module et une icône d'engrenage sont utilisés.

8.3 Le datacore de l'entité

`src/monprojet/ModuleTask/Core/taskCore.json` :

```
{
  "name": "task",
  "label": "Tâche",
  "attribut": {
    "1": { "name": "name", "datatype": "string", "size": "255",
           "nullable": "not", "formtype": "text" },
    "2": { "name": "description", "datatype": "text",
           "nullable": "default", "formtype": "textarea" },
    "3": { "name": "done", "datatype": "integer",
           "nullable": "default", "formtype": "checkbox" }
  },
  "relation": [
    { "entity": "project", "cardinality": "manyToOne",
      "nullable": "default", "ondelete": "cascade" }
  ],
  "actions": [ { "code": "close", "label": "Clôturer" } ]
}
```

Trois détails comptent : `attribut` est un objet indexé (`"1"`, `"2"`…), pas un tableau ; `id` est implicite et ne se déclare jamais ; `actions` déclare les actions métier qui s'ajoutent aux six actions implicites (Tome IV, chapitre 2).

8.4 Générer

```
php devups core:g:moduleendless monprojet\ModuleTask
php devups core:g:frontcontroller monprojet\ModuleTask\task
```

Fichier	Rôle
`Entity/Task.php`	l'entité, avec ses annotations Doctrine
`Form/TaskForm.php`	le formulaire déclaratif
`Datatable/TaskTable.php`	la liste déclarative
`Controller/TaskController.php`	le CRUD de back-office
`Controller/TaskFrontController.php`	l'API REST dédiée
`monprojet.moduletask.php`	les `require`
`index.php`, `services.php`, `.htaccess`, `ModuleTask.php`, `Resource/`	les pièces du module

8.5 Les trois gestes manuels

1. Décommenter le `require` du `FrontController` dans `monprojet.moduletask.php`.
2. Ajouter le module à `src/requires.php` :

```
require 'monprojet/ModuleTask/monprojet.moduletask.php';
```

1. Exposer l'entité à l'API, dans `route/WebService.php` — et décider ce qui est public :

```
public $public_access = [
    // ...
    'task' => \Task::class,
];

public static $publicMethods = [
    // ...
    'task' => ['index', 'detail'],   // lectures anonymes ; tout le reste exige un jeton
];
```

Sans l'entrée dans `$public_access`, `GET /api/task` répond comme une entité inconnue. Sans entrée dans `$publicMethods`, toutes ses méthodes exigent un jeton (Tome III, chapitre 8).

8.6 Créer la table

Écrire une migration, par exemple `database/migration/2026_11_20_create_task.sql`, en partant du SQL proposé par Doctrine :

```
php devups schema:update --dump-sql      # copier le SQL de la nouvelle table
php devups migrate --dry-run             # relire
php devups migrate                       # jouer
php devups dvups_:update                 # l'écran apparaît au back-office
php devups rights:sync                   # ses actions sont attribuables aux rôles
```

8.7 Vérifier

Chemin	Attendu
`/admin/monprojet/ModuleTask/task/list`	la liste, avec ses colonnes
`GET /api/task`	la liste paginée, `{"success":true,...}`
`GET /api/task/1`	le détail
`POST /api/task` avec `{"task":{"name":"x","project.id":3}}` et un jeton	la création
`php devups migrate:status`	aucune migration en attente
`php devups schema:update --dump-sql`	rien à faire

Noter la forme `"project.id": 3` dans la charge utile : le point référence une relation `manyToOne`.

8.8 Si quelque chose ne marche pas

Symptôme	Cause la plus probable
ni `dvups_:update` ni `schema:update` ne voient le module	le `<module>Core.json` manque (§7.3)
`Fatal error: Class TaskController not found`	`src/requires.php` n'a pas été mis à jour
`Cannot redeclare class Task`	double bloc de `require` après une régénération (§6.7)

Symptôme	Cause la plus probable
`Class 'Task' does not exist` alors que le fichier est là	classmap à régénérer : `composer dump-autoload`
l'entité n'apparaît pas au back-office	`php devups dvups_:update` n'a pas été lancé
l'écran répond 403 pour un administrateur qui n'est pas maître	`rights:sync` non lancé, ou privilège non attribué au rôle
`GET /api/task` répond comme une entité inconnue	`$public_access` (§8.5)
`GET /api/task` répond 401 sans jeton	`index` absent de `$publicMethods` — c'est le comportement voulu par défaut
la clé étrangère accepte NULL malgré `"nullable":"not"`	le générateur ignore `nullable` sur une relation (§6.7)

Chapitre 9 — Les tests

9.1 Les trois suites

Le framework est livré avec une suite PHPUnit 10, configurée par `phpunit.xml.dist` :

Suite	Commande	Contenu
unitaire	`composer test:unit`	sans base de données : chargement de `.env`, codes d'activation, propriété, neutralité du dépôt, langues
intégration	`composer test:integration`	contre une base dédiée, recréée à chaque exécution : CRUD du modèle, `QueryBuilder`, droits, migrations, journal, courriels
bout en bout	`composer test:e2e`	un serveur `php -S` démarré automatiquement : API, connexion, sécurité, chaque écran liste et formulaire du back-office
toutes	`composer test`	les trois

L'intégration continue (`.github/workflows/tests.yml`) exécute `composer test` sur PHP 8.1, 8.2 et 8.3 avec un service MySQL, à chaque push et chaque pull request.

9.2 La base de test

La suite utilise une base **dédiée** (`DB_NAME=devups_phpunit` par défaut), supprimée et recréée à chaque exécution par `tests/bootstrap.php`. Par sécurité, le bootstrap refuse toute base dont le nom ne contient pas `test` ou `phpunit` : impossible de vider une base de développement par erreur.

Pour changer les accès, définir `DB_USER`, `DB_PASSWORD`… dans l'environnement, ou copier `phpunit.xml.dist` en `phpunit.xml` (non versionné). Les fichiers de traduction utilisés par les tests vivent dans un dossier temporaire (`LANG_DIR`) : un test ne modifie jamais `lang/fr.php` ni `lang/en.php`.

9.3 Les classes d'appui

Classe	Rôle
`Tests\Support\DatabaseTestCase`	une transaction par test, annulée à la fin
`Tests\Support\HttpServer`	démarre et arrête `php -S` sur le port de test (`APP_PORT`, `:8765` par défaut)
`Tests\Support\HttpTestCase`	requêtes HTTP, session d'administration, jeton JWT

9.4 L'écran par écran

`tests/E2E/AdminScreensTest.php` ouvre chaque écran liste et chaque formulaire de chaque entité déclarée dans `dvups_configurations.php`. Un écran connu comme cassé est inscrit dans `KNOWN_BROKEN` et signalé comme incomplet plutôt qu'en échec. En 7.0, il n'y reste que les deux écrans de journal, dont le formulaire n'existe pas par conception : ces écrans sont en lecture seule.

> **Règle de travail.** Une correction de `dclass/` s'accompagne d'un test qui échoue avant et passe après. Un écran qu'on ajoute au back-office est couvert automatiquement par `AdminScreensTest` dès qu'il figure dans `dvups_configurations.php`.

Fin du Tome I. Le Tome II traite des données : le datacore en détail, les entités, le QueryBuilder, l'écriture, les transactions, les migrations et la traduction des données.